

Računarski sistem - hardver

Za računarski sistem važi opća definicija sistema: **skup međusobno povezanih elemenata koji djeluju ka ostvarenju funkcije cilja**. Karakterišu ga: **struktura** (način na koji su komponente međusobno povezane) i **funkcija** (djelovanje pojedinih komponenti). Pored toga, organizacija računara je hijerarhijska: svaka glavna komponenta se razlaže na podkomponente, a onda podkomponente se opisuju svojom vlastitom strukturom i funkcijom. To znači da se hijerarhijska organizacija računara opisuje odozgo prema dole:

- **Računarski sistem:** osnovne komponente računarskog sistema su: procesor, memorija, U/I;
- **Procesor:** procesorske osnovne komponente su: kontrolna jedinica, registri, aritmetičko-logička i izvršna jedinica;
- **Kontrolna jedinica:** obezbjeđuje kontrolne signale za rad i koordinaciju svih ostalih procesorskih elemenata.

Arhitektura i organizacija

Računarska arhitektura se odnosi na systemske osobine koje su vidljive programeru, odnosno osobine koje imaju direktan uticaj na logičko izvršavanje programa. To su: instrukcijski set, broj bitova za predstavljanje različitih tipova podataka (brojevi, karakteri i sl.), U/I mehanizmi, i tehnike adresiranja memorije.

Računarska organizacija se odnosi na operativne jedinice i njihovo povezivanje kojima se ostvaruju arhitekturne specifikacije. To su: kontrolni signali, različiti interfejsi između računara i periferala, memorijske tehnologije.

Na primjer, pitanje arhitekture je da li će računar imati instrukciju za množenje, a organizacijsko pitanje je da li će ta instrukcija biti implementirana pomoću specijalne jedinice za množenje ili mehanizmom višekratnog korištenja jedinice za sabiranje. U tom slučaju, izbor organizacije može se zasnivati na očekivanoj frekvenciji korištenja instrukcije množenja, relativnim brzinama oba pristupa, te cijeni i fizičkoj veličini jedinice za množenje. Razlika između arhitekture i organizacije je veoma važna.

Struktura i funkcija

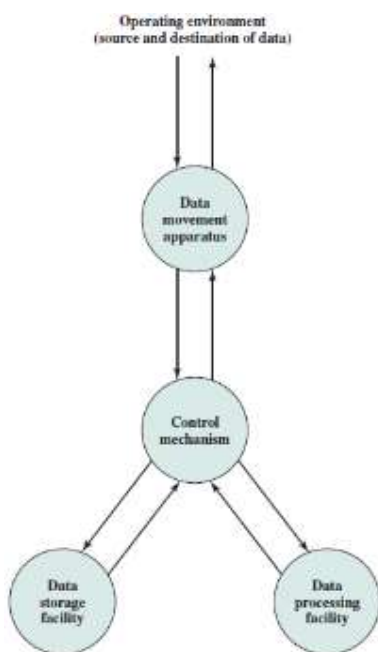
Računar je složen sistem. Sastoji se od miliona osnovnih elektronskih komponenti. Pa kad ih je toliko, kako ih jasno opisati? Ključ rješenja: prepoznati hijerarhijsku prirodu složenih sistema. Hijerarhijski sistem je skup međusobno povezanih podsistema. Hijerarhijska priroda je osnova projektovanje i opis sistema. Projektant samo treba da se bavi određenim nivoom sistema. Na svakom nivou sistem se sastoji od skupa komponenti i njihovih veza. Ponašanje svakog nivoa zavisi samo od pojednostavljenog prikaza sistema na nižem nivou. Na svakom nivou projektant se brine o:

- **Strukturi:** na koji način su komponente međusobno povezane,
- **Funkciji:** djelovanje svake komponente kao dijela strukture.

Funkcija

U osnovi su struktura i funkcija računara jednostavne. Donja slika prikazuje osnovne funkcije računara. Četiri opće funkcije računara:

- obrada podataka
- pohranjivanje podataka
- kretanja podataka
- kontrola.

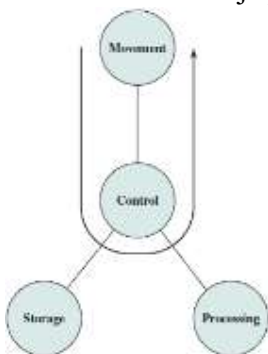


Računar mora biti u stanju da **obrađuje podatke**. Podaci mogu imati različitu formu, ali su i zahtjevi za obradom raznoliki. Međutim, samo je nekoliko osnovnih metoda obrade.

Osnovna funkcija računara je i **pohrana podataka**, čak i u slučaju „obrade u letu“. Šta je „obrada u letu“? Podaci dolaze, obrađuju se i odmah prosljeđuju izvan računara. Računar mora privremeno pohraniti podatke. Zato mora postojati funkcija kratkotrajne pohrane. Također, računar mora pohraniti podatke na duži period. Datoteke se pohranjuju u računar i naknadno se uređuju.

Računar mora obezbijediti **kretanje podataka** između sebe i vanjskog svijeta. Radna okolina računara: uređaji koji služe ili kao izvor ili odredište podataka. *Ulazno-izlazni proces*: računar prima ili prosljeđuje podatke uređaju koji je direktno povezan na njega. Ta se uređaj označava kao *periferal*. *Proces podatkovne komunikacije*: podaci se prenose na veće udaljenosti, sa ili na udaljeni uređaj.

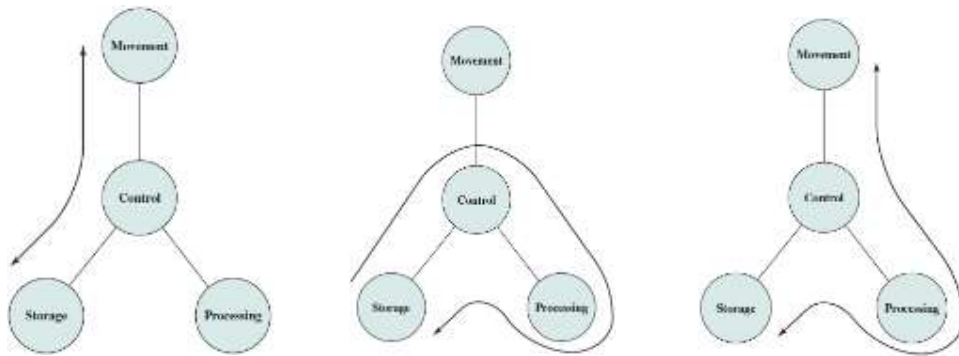
Na kraju, mora postojati **kontrola** gornje tri funkcije. U konačnicu radi je pojedinac (pojedinci) koji obezbjeđuje instrukcije računaru. Unutar računara, kontrolna jedinica upravlja računarskim resursima i usklađuje performanse njegovih funkcionalnih dijelova kao odziv na te instrukcije.



Dakle, na općem nivou svega je nekoliko mogućih operacija. Računar može funkcionirati kao uređaj za prenos podataka (slika lijevo), jednostavno prenoseći podatke od jednog periferala ili komunikacijske linije, do drugog.

Računar funkcionira i kao uređaj za pohranjivanje podataka (donja lijeva slika). Podaci se prenose iz vanjskog svijeta u računarsko skladište (učitavanje) i obrnuto (ispisivanje).

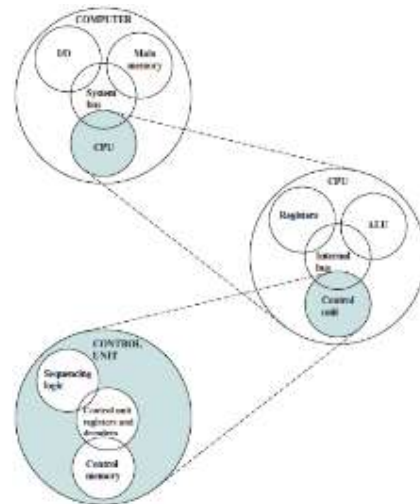
Pored toga, donje slike prikazuju operacije koje uključuju obradu nad podacima iz skladišta (slika u sredini), ili obradu podataka na putu između skladišta i vanjske okoline (slika desno).



Struktura

Računar je u interakciji sa svojom okolinom. Uopće, sve veze sa okolinom mogu se podijeliti na: periferne uređaje i komunikacijske linije. Međutim, za nas je važnija unutrašnja struktura računara (slika desno). Četiri su osnovne komponente strukture:

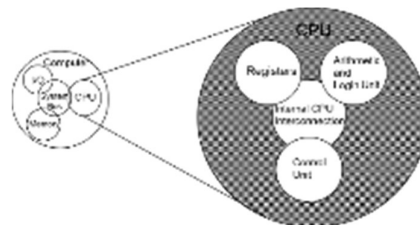
1. **Centralna procesna jedinica (CPU):** Kontrolira rad računara i izvodi funkcije obrade podataka. Često se samo označava kao **procesor**.
2. **Glavna memorija:** Pohranjuje podatke.
3. **U/I:** Prebacivanje podataka između računara i okoline.
4. **Sistemska povezivanje:** Mehanizam povezivanja elemenata računara.



Može biti jedna ili više od svakih gore spomenutih komponenti. Obično je u pitanju samo jedan procesor. U posljednje vrijeme, povećano je korištenje više procesora u jednom računar.

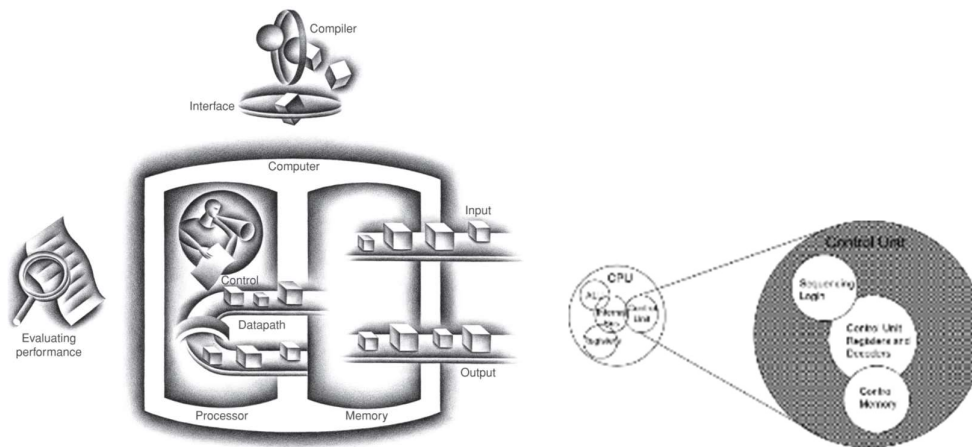
Svakako da je od svih komponenti CPU najzanimljiviji i najsloženiji (donja slika lijevo). Glavne strukturne komponente CPU-a su:

- **Kontrolna jedinica:** kontrolira rad CPU-a
- **Aritmetičko-logička jedinica (ALU):** vrši obradu podataka
- **Registri:** pohrana unutar CPU-a
- **CPU-povezanost:** mehanizam povezivanja elemenata CPU-a.



Unutrašnja struktura računara može se prikazati i na način predstavljen slikom dole lijevo. Procesor dobija podatke i instrukcije iz memorije. U memoriju se podaci učitavaju iz ulaza, a na izlaze se podaci ispisuju iz memorije. Kontrolna jedinica šalje signale koji definišu operacije ALU-a (datapath), memorije, ulaza i izlaza.

Postoji nekoliko pristupa implementacije kontrolne jedinice. Opći je *mikroprogramirana* implementacija. U osnovi, mikroprogramirana jedinica radi izvršavanje mikroinstrukcija koje definišu njenu funkcionalnost (slika gore desno).



Pod poklopcem

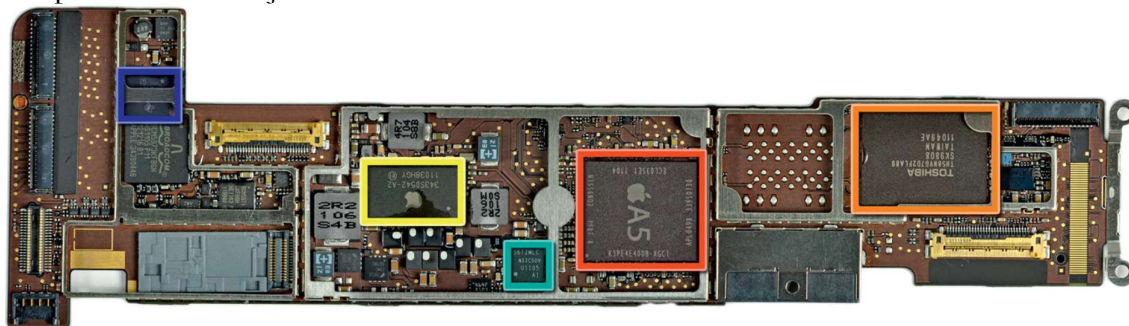
Imajući na umu hijerarhijski prikaz komponenti računara, sad se može „zaviriti pod haubu“ računara. Donja slika prikazuje sadržaj Apple iPad 2 tableta. Pošto je ovo uređaj koji je prvenstveno namijenjen za čitanje, logično je da od četiri osnovne komponente dominiraju ulazno-izlazne, i to: LCD zaslon osjetljiv na dodir, prednja i stražnja kamera, mikروفon, utičnica za slušalice, zvučnici, žiroskop, Wi-Fi, Bluetooth. S druge strane, ostale komponente su veoma male.



U centru gornje slike je metalno kućište iPad-a sa Apple logom na sredini. Iznad njega je LCD zaslon. Krajnje desno je polimerna baterija (3,8V, 25 Wsati) koja se sastoji od tri litijumske ćelije za 10 sati rada. Krajnje lijevo je metalni okvir kojim se LCD pričvršćuje za kućište. Mali dijelovi koji okružuju kućište su ono što smatramo računarom, često su oblika slova L kako bi smjestili uz bateriju. Donja slika (u boji) bliže prikazuje ploču L-oblika koja se nalazi s donje lijeve strane kućišta. To je štampana ploča sa procesorom i memorijom. Mali pravougaonik

ispod štampane ploče sadrži čip koji obezbeđuje bežičnu komunikaciju: Wi-Fi, bluetooth i FM radio. Smješta se u donji lijevi ugao štampane ploče.

Iznad gornjeg lijevog ugla kućišta je još jedna L-komponenta koja sadrži prednju kameru, utičnicu za slušalice i mikrofoni. Blizu gornjeg desnog ugla kućišta je ploča sa kontrolom jačine zvuka, dugme za zaključavanje rotacije ekrana, te žiroskop. Mali pravougaonik uz ovu ploču je stražnja kamera. Uz donji desni ugao kućišta je sklop kojeg čine zvučnici, također L-oblika. Kabl na dnu povezuje štampanu ploču sa pločom koja sadrži kameru. Ploča između kabla i sklopa sa zvučnicima je kontroler ekrana na dodir.



Na gornjoj slici uokvireno je pet integralnih kola. Veliko integralno kolo u sredini je Apple A5 čip, koji sadrži dvije ARM procesorske jezgre koje rade na 1GHz, zajedno sa 512 MB glavne memorije. Donja slika je fotografija unutrašnjosti A5 čipa.



Veličina čipa je 12,1 x 10,1 mm. Sadrži i PowerVR grafički procesor (GPU) sa četiri aritmetičke jedinice u gornjem lijevom dijelu. Lijevo i dole su interfejsi prema glavnoj memoriji (DRAM).

Čip, slične veličine na gornjoj slici u boji, je 32 GB fleš memorijski čip za neizbrisivo pohranjivanje. Prazan je prostor između ova dva čipa gdje se može ugraditi drugi fleš memorijski čip za udvostručenje kapaciteta. Ostali čipovi uključuju kontrolere struje i U/I.

Šta se nalazi ispod programa

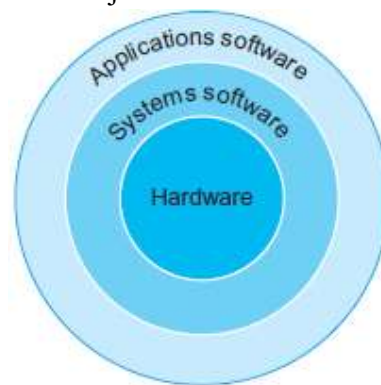
Kad smo zavirili pod haubu i vidjeli kakvom mašinom raspolažemo, vrijeme je da se upoznamo sa upravljačkim mehanizmom. Tipična aplikacija za obradu teksta ili veliki sistem baze podataka, može se sastojati od miliona linija koda i oslanjati se na sofisticirane softverske biblioteke koje implementiraju složene funkcije kao podršku za aplikacije. Međutim, računarski hardver, koji smo gore opisali, može izvršavati samo veoma jednostavne instrukcije niskog nivoa. Kako bi se došlo sa složenih aplikacija, tipa obrade teksta ili baze podataka, na jednostavne instrukcije, uključuje se nekoliko nivoa softvera koji prevode (interpretiraju) operacije visokog nivoa u jednostavne kompjuterske operacije.

Donja slika prikazuje ove softverske nivoe koji su organizovani na već spomenuti hijerarhijski način. Aplikacije čine vanjski prsten, a različiti sistemski softveri su smješteni između hardvera i aplikacijskog softvera.

Mnogo je tipova sistemskih softvera, ali dva danas su najvažnija i zauzimaju centralno mjesto u svakom današnjem računaru: operativni sistem i kompajler.

Operativni sistem je sučelje između korisničkog programa i hardvera, a obezbjeđuje različite usluge i funkcije nadzora. Primjeri operativnih sistema današnjice su Linux, iOS i Windows. Najvažnije funkcije operativnog sistema su:

- upravljanje osnovnim operacijama ulaza i izlaza
- alokacija memorije (glavne i sekundarne)
- obezbjeđivanje zaštite dijeljenja resursa računara među konkurentnim aplikacijama koje ih koriste simultano.



Kompajleri imaju još jednu bitnu funkciju: prevode programe visokog nivoa (C, C++, Java, Visual Basic) u instrukcije koje može izvršiti hardver. S obzirom na sofisticiranost modernih programskih jezika visokog nivoa i jednostavnost instrukcija izvršava hardver, prevođenje programskog jezika visokog nivoa u hardverske instrukcije je složeno.

Od jezika visokog nivoa do jezika hardvera

Da bismo stvarno komunicirali sa hardverom, moramo poslati električne signale. Najjednostavniji signali koje razumije računar su *uključiti* (uključiti) i *isključiti* (off), pa alfabet računara čine samo dva slova. Isto kao što i 30 slova našeg alfabeta nas ne ograničava u pisanju, dva slova računarskog alfabeta ne ograničavaju šta računari mogu uraditi. Simboli ova dva slova su 0 i 1, i obično zamišljamo računarski jezik kao brojeve sa bazom 2 (binarni brojevi). Svako „slovo“ ovog alfabeta se označava koa binarna cifra ili bit. Računari izvršavaju naše naredbe koje nazivamo instrukcijama. Instrukcije, koje nisu ništa drugo do kolekcije bitova koje računar razumije i povinjava im se, mogu se smatrati brojevima. Na primjer, bitovi 1000110010100000 kažu računaru da sabere dva broja. Zašto koristimo brojeve za instrukcije i podatke, govorićemo kasnije, ali korištenje brojeva i za instrukcije i za podatke je osnova kompjutinga.

Prvi su programeri komunicirali sa računarima preko binarnih brojeva, ali je ovo bilo tako zamorno da su veoma brzo izumili nove načine obilježavanja koji su mnogo bliže načinu kako ljudi razmišljaju. U početku, ove notacije su ručno prevođenje u binarnu formu, te je proces i dalje bio zamoran. Korištenjem računara u svrhu pomaganja programiranja računara, pioniri su izumili programe za prevođenje simboličkog zapisa u binarni. Prvi od ovih programa nazvan je asembler. On prevodi simbolički oblik instrukcije u binarni. Na primjer, programer napiše

add A, B

što asembler prevodi u: 1000110010100000

Ova instrukcija kaže računaru da sabere dva broja A i B. Ime asemblerski jezik i danas se koristi za ovaj simbolički jezik. S druge strane, binarni jezik kojeg razumije mašina, je mašinski jezik. Asembler od programera zahtijeva da razmišlja kao računar.

Spoznaja da bi se mogao napisati program za prevođenje jezika visokog nivoa u računarske instrukcije bila je jedno od najvećih otkrića u prvim danima kompjuetinga. Današnji programeri svoju produktivnost duguju stvaranju programskih jezika visokog nivoa i kompajlera koji prevode programe takvih jezika u instrukcije (slika desno). Iako je na ovoj slici prikazano prevođenje u dva koraka, neki kompajleri nemaju posrednika i direktno proizvode program u mašinskom jeziku (MIPS - Microprocessor without Interlocked Pipeline Stages).

High-level
language
program
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
    mult $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

Assembler

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
10001100001001000000000000000100
10101110000100100000000000000000
10101101111000100000000000000100
0000001111100000000000000001000
```

Kompajler omogućava programeru da u programu visokog nivoa napiše $A + B$. Kompajler ovo pretvara u iskaz u asemblerskom jeziku `add A, B`. Prema slici desno, asembler bi preveo ovaj iskaz u binarne instrukcije koje kažu računaru da sabere dva broja A i B.

Jezici visokog nivoa imaju nekoliko prednosti. Prvo, dopuštaju programeru da razmišlja na „prirodnijem“ jeziku, korištenjem riječi engleskog jezika i algebarske notacije. Rezultat je program koji više liči tekstu nego tabele kriptiranih simbola. Povrh toga, omogućavaju da jezici budu kreirani s obzirom na njihovu namjenu. Zato je na primjer Fortran kreiran za naučne obrade, Cobol poslovnu obradu podataka, Lisp za rad sa simbolima itd.

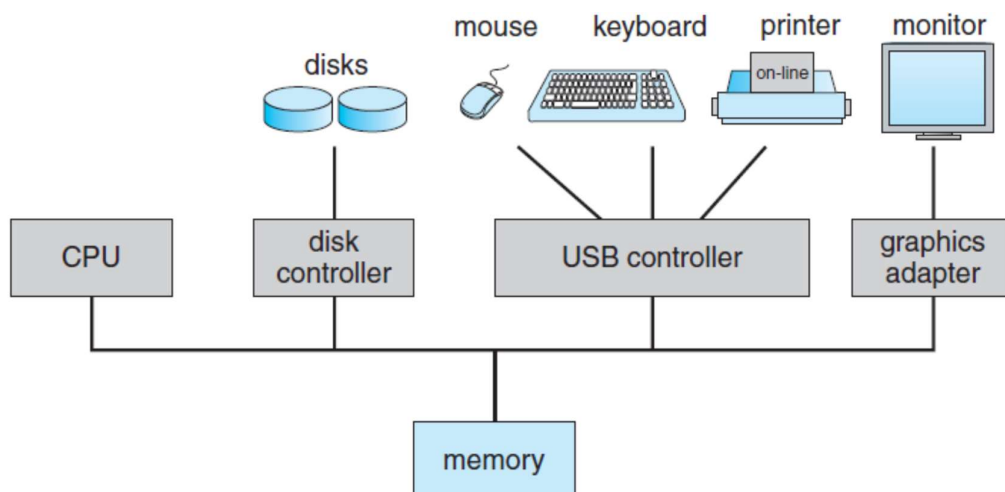
Druga prednost jezika visokog nivoa je unapređenje produktivnosti programera. Jedno od rijetkih slaganja među onima koji se bave razvojem softvera, je da je potrebno manje vremena za razvoj programa u jezicima koji zahtijevaju manje linija za izražavanje ideje. Sažetost je očigledna prednost jezika visokog nivoa u odnosu na asembler.

Posljednja prednost je što jezici visokog nivoa dopuštaju da programi budu nezavisni od računara na kojem su razvijani, pošto kompajleri i asembleri mogu prevoditi programe jezika visokog nivoa u binarne instrukcije bilo kojeg računara. Ove tri prednosti su tako velike da se toliko malo programira u asembleru.

Kratak prikaz računarskog hardvera

Moderni računarski sistemi opće namjene sastoje se od jednog ili više CPU-a i brojnih kontrolera uređaja. Povezani su preko zajedničke sabirnice koja obezbjeđuje pristup djeljivoj memoriji (donja slika). Kontroleri uređaja su odgovorni za određene uređaje kao što su diskovi, audio ili video uređaji itd. CPU i kontroleri uređaja mogu izvršavati poslove paralelno „takmičeći“ se međusobno za pristup memoriji. Radi osiguravanja propisnog pristupa djeljivoj memoriji, memorijski kontroler sinhronizira pristup memoriji.

Da bi se računar pokrenuo mora se izvršiti početni program tzv. *bootstrap*. Težnja je da ovaj program bude jednostavan. Obično je pohranjen u ROM-u ili EEPROM-u. Također, naziva se i *firmware*. On inicijalizira sve dijelove sistema, od CPU registara, preko kontrolera uređaja, do sadržaja memorije. Bootstrap mora znati kako učitati operativni sistem, te kako započeti izvršavanje sistema. Da bi ispunio ovaj cilj, bootstrap mora locirati jezgro operativnog sistema i učitati ga u memoriju.



Kad je jezgro učitano, počinje sa pružanjem usluga sistemu i svojim korisnicima. Po završetku ove faze sistem je u potpunosti pokrenut i čeka na izvršavanje poslova.

Procesori

„Mozak“ računara je CPU. On dobavlja instrukcije iz memorije i izvršava ih. Osnovni ciklus svakog CPU-a je prihvatanje instrukcije iz memorije, dekodiranje radi određivanja njenog tipa i operanada, izvršavanje, te nakon toga dohvatanja sljedeće instrukcije. Ciklus se ponavlja do završetka programa. Na ovaj način se izvršava program.

Pošto pristup memoriji zbog dobavljanja instrukcija ili podataka traje duže nego izvršavanje instrukcije, svaki CPU sadrži registre u kojima čuva ključne varijable i privremene rezultate. Zato skup instrukcija u općem slučaju sadrži instrukcije za prebacivanje riječi iz memorije u registre, i obrnuto, iz registara u memoriju. Ostale instrukcije udružuju dva operanda iz registara, memorije, ili iz oboje, u rezultat, kao što je sabiranje dvije riječi i pohranjivanje rezultata u registar ili memoriju.

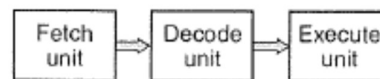
Pored registara opće namjene koji se koriste za držanje varijabli i privremenih rezultata, većina računara ima nekoliko specijalnih registara. Jedan od njih je programski brojač, koji sadrži memorijsku adresu sljedeće instrukcije koja će biti dobavljena. Nakon njenog prihvatanja, programski brojač se ažurira kako bi pokazao na sljedeću.

Sljedeći tip je stek pokazivač, koji pokazuje na vrh trenutnog memorijskog steka. On sadrži okvir za svaku proceduru koja je ušla ali nije izašla. Okvir stek-procedure sadrži one ulazne parametre, lokalne i privremene varijable koje se ne čuvaju u registrima.

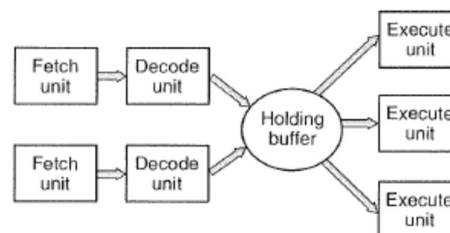
Još jedan od tipova registara je PSW (Program Status Word) koji sadrži bitove stanja. Ovi se bitovi postavljaju na osnovu instrukcija upoređivanja, CPU prioriteta, korisničkog ili kernel moda, i ostalih različitih kontrolnih bitova. Korisnički program u normalnim uslovima može čitati cijeli PSW, ali može upisivati samo u neka njegova polja. PSW ima presudnu ulogu kod sistemskih poziva i I/O.

Operativni sistem mora biti „svjestan“ svih registara. Kod multipleksiranja CPU-a, operativni sistem često zaustavlja pokrenuti program kako bi započeo ili nastavio drugi. Svaki put kada zaustavi njegovo izvršavanje, operativni sistem mora sačuvati sadržaj svih registara kako bi mogao povratiti njihovo stanje kad se program kasnije nastavi.

Projektanti CPU-a su davno napustili jednostavni model prihvatanja, dekodiranja i izvršavanja jedne instrukcije, a sve u cilja poboljšanja performansi. Moderni CPU ima svojstva izvršavanja više od jedne instrukcije istovremeno. Na primjer, CPU može imati odvojene jedinice za prihvatanje, dekodiranje i izvršavanje, tako da dok izvršava n -tu instrukciju istovremeno može dekodirati $n+1$, a dohvatiti $n+2$. Ovakva organizacija se naziva pipeline, što je predstavljeno na slici desno (pipeline sa tri faze, mada su česti i duži).



Naprednija izvedba od pipeline-a je superskalar CPU (slika desno). Kod ove izvedbe postoji više izvršnih jedinica, na primjer, jedna za rad sa integer-ima, druga za float-point, i jedna za Bulove operacije. Istovremeno se prihvata dvije ili više instrukcija, i ostavljaju u buffer do njihovog izvršavanja. Čim je izvršna jedinica slobodna, pregledava se ovaj buffer, te ako se u njemu nalazi instrukcija, ona se iz njega pomiče na izvršenje.



Većina CPU-a, osim veoma jednostavnih za ugrađene sisteme (embedded), imaju dva moda, kernel i korisnički. Obično bit u PSW-u kontroliše mod. Kad je u kernel modu, CPU može izvršiti svaku instrukciju iz instrukcijskog skupa i koristiti sva svojstva hardvera. Operativni sistem se izvršava u kernel modu, zbog čega ima pristup cijelom hardveru. Naravno da korisničkom programu nije dozvoljeno postavljanje PSW-bita moda unošenjem kernel moda. Da bi dobio usluge od operativnog sistema, korisnički program mora napraviti sistemski poziv kojeg hvata kernel i poziva operativni sistem. TRAP instrukcija prebacuje iz korisničkog u kernel mod i pokreće operativni sistem. Kad se posao izvrši, kontrola se vraća korisničkom programu instrukcijom koja prati sistemski poziv.

Donedavno, većina računarskih sistema je bila sa jednim procesorom. Kod ovakvih sistema postoji jedan glavni CPU koji izvršava skup općih instrukcija uključujući instrukcije korisničkih procesa. Skoro svi jednoprocesorski sistemi imaju i procesore za posebne namjene. Mogu doći

kao procesori odgovarajućih uređaja kao što su U/I procesori koji veoma brzo prebacuju podatke između sistemskih komponenti.

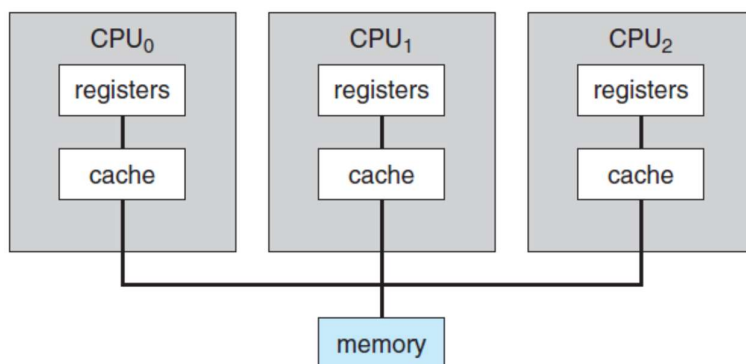
Ovi procesori za posebne namjene izvršavaju ograničen skup instrukcija među kojima nisu korisnički procesi. Npr., mikroprocesor kontrole diska prima zahtjeve od glavnog procesora i sprovodi ih vlastitim algoritmom redanja i raspoređivanja. Na ovaj način, glavni procesor je oslobođen posla koji se tiče upravljanja diskom. Uobičajeno je korištenje mikroprocesora posebne namjene, ali to ne znači da jednoprocesorski sistem postaje višeprocesorski.

Posljednjih nekoliko godina dominiraju *višeprocesorski* ili *paralelni* sistemi. Takvi sistemi imaju dva ili više procesora koji usko sarađuju i dijele sabirnicu, a ponekad i takt (clock), memoriju i periferne uređaje. Prvo su se pojavili kod servera, a onda i kod desktop računara i laptopa. U posljednje vrijeme koriste se i u mobilnim uređajima kao što su pametni telefoni i tableti.

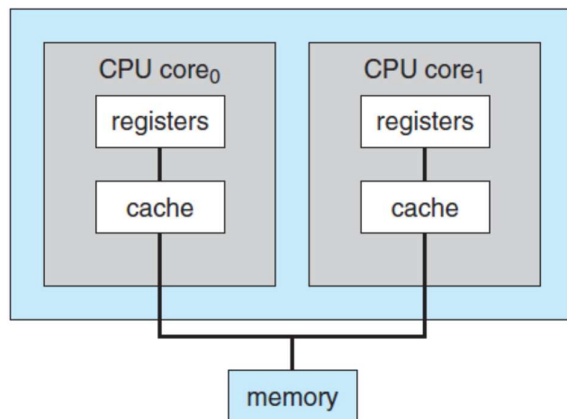
Višeprocesorski sistemi imaju tri prednosti:

1. **Povećana propusnost.** Povećanjem broja procesora očekivati je da veća količina rada može biti urađena za kraće vrijeme. Međutim, povećanje brzine nije proporcionalno povećanju broja procesora, već je manji. Kada više procesora zajedno rade na zadatku, određena količina rada se utroši na usklađivanje njihovog rada. Pored toga, dijeljenje istih resursa smanjuje teoretski očekivanu efikasnost. Ova situacija je slična slučaju kada N programera rade zajedno; vrijeme koje im je potrebno za rješenje problema nije N puta manje od onog koje je potrebno samo jednom programeru da uradi isti posao.
2. **Ekonomičnost.** Višeprocesorski sistemi obično su jeftiniji nego ekvivalentni veći broj jednoprocesorskih sistema jer dijele periferale, uređaje za pohranjivanje i napajanje. Ako nekoliko programa rade sa istim skupom podataka, jeftinije ih je pohraniti na jedan disk odakle ih koriste svi procesori, nego posjedovati više računara sa vlastitim diskovima i više kopija podataka.
3. **Povećana pouzdanost.** Ako funkcije mogu biti pravilno raspoređene na nekoliko procesora, onda otkaz jednog neće zaustaviti sistem, već će ga samo usporiti. Npr., ako u sistemu sa deset procesora jedan otkáže, onda preostali preuzimaju posao procesora u otkazu. Zato, teoretski, sistem radi samo 10% sporije, ali nije u cijelosti u otkazu.

Današnji višeprocesorski sistemi mogu biti jedan od dva sljedeća tipa. Neki sistemi koriste *asimetrično multiprocesiranje* kod kojeg se svakom procesoru dodjeljuje poseban zadatak. Glavni procesor kontroliše sistem, dok drugi procesori ili dobijaju instrukcije od glavnog, ili imaju predefinisane zadatke. Međutim, najčešće se koristi *simetrično multiprocesiranje* (SMP) kod kojeg svaki procesor izvršava sve zadatke unutar operativnog sistema. To znači da su svi procesori istog statusa (donja slika). Treba primijetiti da svaki procesor ima vlastiti skup registara i keš. Međutim, svi procesori dijele fizičku memoriju.



U posljednje vrijeme trend u dizajniranju CPU-a podrazumijeva više računarskih jezgri na jednom čipu. Takvi višeprosesorski sistemi se označavaju kao *višejezgreni*. Mogu biti efikasniji nego više procesora sa jednom jezgrom jer komunikacija preko čipa je brža nego između čipova. Pored toga, jedan čip sa više jezgri treba znatno manje energije nego više čipova sa jednom jezgrom. Na donjoj slici prikazan je dizajn čipa sa dvije jezgre (dual-core).

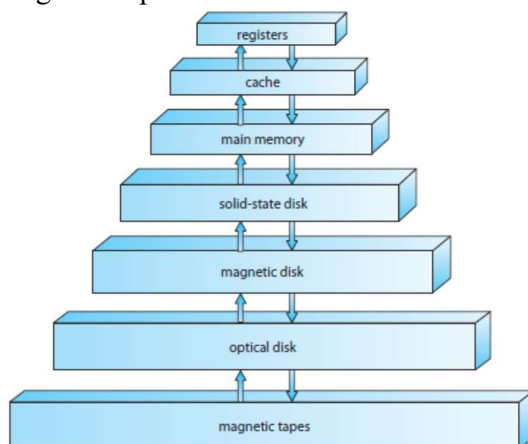


Memorija

Sljedeća važna komponenta računara je memorija. Idealno bi bilo da je memorija ekstremno brza (brža od izvršavanja instrukcije tako da CPU ne čeka), bezgranično velika i veoma jeftina. Nijedna trenutna tehnologija ne može zadovoljiti ove uslove, te se koristi drugačiji pristup. Zato je memorijski sistem konstruisan u obliku hijerarhijskih nivoa (donja slika). Veći nivoi imaju veću brzinu, manji kapacitet, i veću cijenu po bitu nego oni ispod.

Nivo na vrhu se sastoji od registara smještenih u CPU. Načinjeni su od istog materijala kao i CPU, pa su iste brzine kao i on. Zato nema kašnjenja pri njihovom pristupanju. Njihov kapacitet je obično 32x32-bita na 32-bitnom CPU i 64x64-bita na 64-bitnom. Manje od 1KB u oba slučaja.

Sljedeća je keš memorija. Kada program treba pročitati memorijsku riječ, keš hardver provjerava da li je potrebna linija u kešu. Ako jeste (što se naziva cache hit), tražena linija se uzima iz keša, te nisu potrebni dodatni memorijski zahtjevi koji se preko sabirnice šalju glavnoj memoriji. Za cache hit su obično potrebna dva ciklusa. Ukoliko linija nije u kešu (cache miss) potrebno je dodatno vrijeme potrebno za pristup glavnoj memoriji. Keš memorija je ograničene veličine zbog velikih troškova.



Keširanje igra glavnu ulogu u mnogim područjima kompjuterske nauke, a ne samo keširanje linija RAM-a. Kad god veliki resurs može biti podijeljen u dijelove, od kojih se neki koriste mnogo više od drugih, koristi se keširanje za poboljšanje svojstava. Operativni sistemi ga uvijek koriste. Na primjer, većina operativnih sistema drži dio veoma često korištenih fajlova u glavnoj memoriji kako bi se izbjeglo njihovo ponovljeno dohvaćanje sa diska. Slično, rezultati pretvaranja dugih imenskih putanja, kao `/home/ast/projects/minix3/src/kernel/clock.c` u disk adrese na

kojima je smješten fajl, mogu biti keširani kako bi se izbjegli ponovljeni upiti. Pored toga, kad se adresa Web stranice (URL) pretvara u mrežnu (IP), rezultat može biti keširan za buduće korištenje.

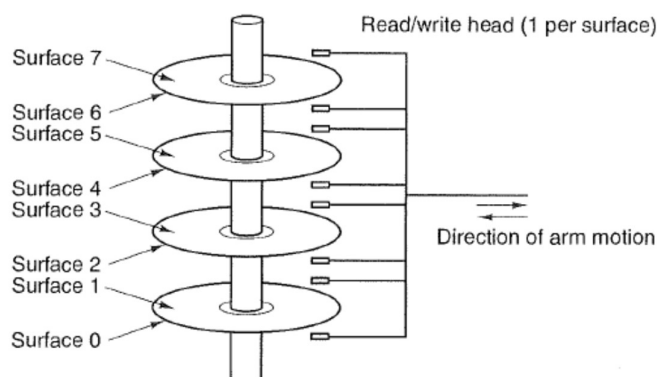
Računari opće namjene izvršavaju većinu svojih programa iz glavne memorije (RAM – Random Access Memory), koja je sljedeća u hijerarhiji memorija. Današnje izvedbe su kapaciteta od nekoliko gigabajta, i rapidno rastu. Obično se implementira kao poluprovodnička i naziva se dinamički RAM (dynamic random access memory – DRAM).

Pored glavne memorije računari posjeduju malu količinu nepromjenjivog ROM-a. Za razliku od RAM-a nepromjenjiva memorija ne gubi svoj sadržaj isključivanjem. ROM (Read Only Memory) je fabrički programiran i poslije se ne može mijenjati, to jest, sadrži samo statičke programe kao što je bootstrap (početni) program. Također, neke I/O kartice imaju ROM za rukovanje kontrolom uređaja na niskom nivou. EEPROM (Electrically Erasable Programmable ROM) za razliku od ROM-a se može brisati i ponovo upisivati, ali ne često, pa sadrži uglavnom statičke programe.

Diskovi

Sljedeći u hijerarhiji je magnetni (tvrdi) disk. Disk je dva reda veličina jeftiniji od RAM-a po bitu, i dva reda veličina veći. Jedini je problem što je vrijeme pristupa skoro tri reda veličina sporije. Ovako mala brzina je posljedica činjenice da je disk mehanički uređaj (donja slika).

Disk se sastoji od jedne ili više ploča koje se vrte brzinom 5400, 7200 ili 10800 obrtaja u minuti. Mehanička ruka se kreće iznad diskova slično nosaču igle na gramofonima. Informacije na diskovima su zapisane u serije koncentričnih krugova. U bilo kojem položaju ruke, svaka glava može čitati kružni dio koji se zove traka. Sve trake zajedno za dati položaj ruke čine cilindar.



Svaka je traka podijeljena u nekoliko sektora čija je veličina obično 512 bajta. Kod novijih diskova vanjski cilindri imaju više sektora od unutrašnjih. Pomicanje ruke između dva susjedna cilindra traje oko 1 ms. Njeno nasumično micanje obično traje 5-10 ms zavisno od diska. Kad je ruka na odgovarajućoj traci dodatno je potrebno 5-10 ms dok odgovarajući sektor dođe pod glavu, što zavisi od brzine obrtanja diska. Čitanje (pisanje) se vrši brzinom od 50 MB/s kod sporih, 160 MB/s kod brzih diskova.

Mnogi računari podržavaju tzv. virtualnu memoriju. Ona omogućava pokretanje programa koji su veći od fizičke memorije, smještajući ih na disk, dok se glavna memorija koristi kao neka vrsta keša za dijelove koji se veoma često izvršavaju.

Trake

Ovaj se medij često koristi kao backup diskova i čuvanje velike količine podataka. Kako bi se pristupilo podacima na njima, moraju biti prvo fizički umetnute u čitač trake, što radi ili čovjek ili robot. Nakon toga, traka mora biti premotana na zahtijevani blok, za što treba i po nekoliko minuta. Velika prednost traka je što su veoma jeftine po jedinici memorije i izmjenjive. To je veoma važno za backup-trake koje moraju biti pohranjene izvan mjesta korištenja zbog požara, poplave, zemljotresa i sl.

I/O uređaji

CPU i memorija nisu jedina sredstva kojima operativni sistem upravlja. I I/O uređaji su usko vezani sa operativnim sistemom. I/O uređaji se u općem slučaju sastoje od kontrolera i samog uređaja. Kontroler je jedan ili više čipova koji fizički kontrolira uređaj. Prihvata komande od operativnog sistema i izvodi ih. Pošto je svaki tip kontrolera različit, to je za njihovu kontrolu potreban i različit softver. Softver koji „razgovara“ sa kontrolerom izdavajući mu komande i prihvatajući njegove odzive zove se upravljački program (device driver).

Sabirnica

Sabirnica je skup provodnika i konektora preko kojih se prenose podaci. Ona se često upoređuje sa autoputom po kojem podaci putuju od jedne do druge komponente unutar računara. Računarski sistemi sadrže brojne i različite sabirnice koje omogućavaju putanje između komponenti različitog nivoa hijerarhije računarskog sistema. Sabirnica koja povezuje glavne elemente (procesor, memorija, I/O) zove se **sistemska sabirnica**. Najčešće strukture veza računara se zasnivaju na korištenju jedne ili više sistemskih sabirnica.

Sabirnica se sastoji iz tri dijela. To su adresna, podatkovna i kontrolna sabirnica. Podatkovna prenosi stvarne podatke, a adresna sabirnica prenosi informacije o podacima i gdje trebaju biti poslani. Adresna se koristi za identifikiranje određenih lokacija (adresa) u glavnoj memoriji. Kontrolna sabirnica se koristi za kontrolu pristupa i korištenja adresne i podatkovne.

Literatura

1. Andrew S. Tanenbaum, *Modern Operating Systems 3rd Edition*, Pearson Education Inc., 2008.
2. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*, John Wiley & Sons, Inc., Copyright © 2013, 2012, 2008.
3. William Stallings, *COMPUTER ORGANIZATION AND ARCHITECTURE, DESIGNING FOR PERFORMANCE NINTH EDITION*, Copyright © 2013, 2010, 2006 by Pearson Education, Inc., Upper Saddle River, New Jersey, 07458.
4. David A. Patterson, John L. Hennessy, *COMPUTER ORGANIZATION AND DESIGN, THE HARDWARE/SOFTWARE INTERFACE, FIFTH EDITION*, Copyright © 2014 Elsevier Inc.